Protocol engine — API reference

Public interface of the protocol and hypothesis engine. Intended for the GUI module and for anyone reusing the analysis without reading the implementation. The normalized capture contract is defined in docs/CONTRACT.md and docs/schemas/capture.schema.json; this module does not change it.

1. Public API

All functions and types below are importable from the package paths shown. The GUI works with apply_rule / apply_rule_fields, build_result, verify_on_corpus, ResultStore and compare_reports.

Rules

- src.protocol.rule.parse_rule(raw: dict) -> Rule — build a Rule from a parsed JSON/YAML object. Raises RuleError on a malformed rule.
- src.protocol.rule.load_rule(path: str) -> Rule — read JSON or YAML from disk and parse it.
- src.protocol.rule.Rule — dataclass: schema_version, rule_id, rule_version, name, scope, framing, fields.
- .to_dict(), .bump_version(**changes) -> Rule (returns a copy with rule_version + 1), .direction, .applies_to(direction), .is_out_of_date(result_rule_version).
- src.protocol.rule.FieldSpec — dataclass: name, offset, type, byte_order, length, enum, hypothesis, expected, missing_ok.

Framing and application

- src.protocol.framing.FramingStrategy.from_dict(spec: dict) -> FramingStrategy — parse the framing block. Raises FramingError.
- src.protocol.framing.frame_stream(stream: bytes | DirectionalStream, strategy: FramingStrategy) -> list[Message] — split a byte stream into candidate messages. Message has offset, length, complete, reason.
- src.protocol.engine.apply_rule(stream: DirectionalStream | bytes, rule: Rule, session_id: str = "", direction: str = "A_to_B") -> list[MessageResult] — frame the stream and decode every declared field for every message. When the rule scope excludes the direction, every message is not_applicable.
- src.protocol.engine.apply_rule_fields(...) -> list[FieldResult] — same, but a flat list of per-field results.
- src.protocol.engine.flatten_fields(results: list[MessageResult]) -> list[FieldResult].
- MessageResult — offset, length, status, fields, complete, session_id, direction, reason, bytes_hex; .end, .field_values(), .counterexample().
- FieldResult — message_offset, message_length, field_name, field_type, field_offset, field_length, value, status, hypothesis, provenance_range, session_id, direction, reason.
- Counterexample — status, reason, session_id, direction, message_offset, message_length, field_name, field_offset, field_length, bytes_hex, provenance_range.

Capture input

- src.protocol.stream.load_capture(path: str) -> Capture — read a normalized capture (contract format) from disk.

- `src.protocol.stream.capture_from_dict(raw: dict) -> Capture` — build from a parsed object.
- `Capture.stream(session_id: str, direction: str) -> DirectionalStream`; `Capture.iter_streams() -> list[tuple[str, str, DirectionalStream]]`; `Capture.capture_hash`.
- `src.protocol.stream.DirectionalStream.from_bytes(data: bytes)`, `.from_direction(direction: dict)`, `.holes(start, end)`, `.has_gap(...)`, `.has_ambiguity(...)`, `.provenance`.

Result

- `src.protocol.result.build_result(rule: Rule, capture_id: str, messages: list[MessageResult]) -> ApplicationResult` — build the contract result object.
- `ApplicationResult.to_dict()`; `write_result(result, path, validate=True)`; `validate_result(payload, schema_path=None)`; `message_status(result_payload: dict, offset: int) -> Status | None`. `write_result` selects the format from the path suffix: `.jsonl` writes one compact message per line and `.csv` writes a header plus one row per message with a column per declared field; any other suffix is contract JSON, which is validated against the schema.

Hypothesis

- `src.hypothesis.corpus.CorpusStream.from_bytes(data, session_id="", direction="A_to_B")` — wrap a raw stream for corpus verification.
- `src.hypothesis.corpus.verify_on_corpus(rule: Rule, corpus: list[CorpusStream | DirectionalStream]) -> VerificationReport` — apply the rule to every stream and summarise. The report has `rule_id, rule_version, totals, total, contradictions, messages, summary`, plus `.counts()`, `.is_relevant()`, `.is_confirmed()`, `.to_dict()`. `is_confirmed()` is true only when the rule reached at least one applicable message and nothing contradicted it; a report that is applicable nowhere is not a confirmation. `src.protocol.cli.report_from_dict(payload) -> VerificationReport` rebuilds a report from a stored `verify` JSON document, so `diff`, `metrics` and `alternatives` can read a report without re-running the rule.
- `src.hypothesis.diff.diff_rules(rule_a, rule_b) -> RuleDiff` — added, removed and changed fields, framing and scope changes. Each field-level entry is a `FieldChange` (`name`, `kind`, `before`, `after`). `.to_dict()`; `format_rule_diff(diff) -> str`.
- `src.hypothesis.diff.diff_reports(report_a, report_b) -> ReportDiff` — `resolved_counterexamples, introduced_counterexamples, unchanged_mismatches, matched_delta, coverage_delta, version_a, version_b`. `.to_dict()`; `format_report_diff(diff) -> str`.
- `src.hypothesis.metrics.compute_metrics(report) -> RuleMetrics` — see the metrics section below.
- `src.hypothesis.journal.load_journal(path) -> list[JournalEntry]`, `parse_journal(text)`, `correlate(messages, entries, window_ms=500) -> CorrelationReport` — see the journal section below.
- `src.hypothesis.alternatives.suggest_alternatives(rule, messages, correlations=None) -> AlternativesReport`, `analyze_field(rule, messages, field_name, correlations=None) -> FieldAlternatives` — see the alternatives section below.
- `src.hypothesis.alternatives.entropy(field_values) -> float`, `periodicity(field_values) -> int | None`, `bit_pattern(field_values, bit_count=4) -> dict`, `delta_correlation(field_values) -> dict` — value-shape readings; see the alternatives section below.
- `src.hypothesis.narrative.render_hypothesis_narrative(investigation) -> str` — write the hypotheses of an investigation dictionary (the shape in `src/report/model.py`) as Markdown, filled from the figures already present.
- `src.hypothesis.versioning.ResultStore` — `.add(rule, capture_id, payload) -> StoredResult`, `.results_for(rule_id, capture_id=None)`, `.mark_outdated(rule, capture_id=None) -> list[StoredResult]`, `.to_list()`, `.save(path)`.

- `src.hypothesis.versioning.compare_reports(base, new) -> VersionComparison` — `rule_id`, `base_version`, `new_version`, `base_counts`, `new_counts`, `introduced`, `resolved`.
- `src.hypothesis.status.Status` — enum of all statuses; `CORE_STATUSES`, `ALL_STATUSES`.

Command line

```
python -m src.protocol.cli apply  --rule rule.json --capture capture.json --out result.json
python -m src.protocol.cli verify --rule rule.json --capture capture.json --out report.json
python -m src.protocol.cli diff   --rule-a v1.json --rule-b v2.json --report-a r1.json --report-b r2.json
python -m src.protocol.cli metrics --report r1.json --out metrics.json
python -m src.protocol.cli correlate --capture capture.json --journal journal.md --out correlations.json
python -m src.protocol.cli pair   --rule rule.json --capture capture.json --out pairing.json
python -m src.protocol.cli export --rule rule.json --format kaitai --out out/
python -m src.protocol.cli alternatives --report r1.json --corpus capture.json --journal journal.md --field value --out alternatives.json
```

Each command prints a human-readable summary to stderr and the machine-readable document to stdout, or to `--out` when given.

Errors

Parsing and loading raise a typed `ValueError` subclass, so a caller can distinguish a bad input from an internal fault:

- `src.protocol.rule.RuleError` — `parse_rule` / `load_rule` on a malformed rule.
- `src.protocol.framing.FramingError` — `FramingStrategy.from_dict` on a bad framing block, or framing a stream that cannot satisfy it.
- `src.protocol.stream.CaptureError` — `load_capture` / `capture_from_dict` on a capture that is not in the contract format.
- `src.hypothesis.journal.JournalError` — `load_journal` / `parse_journal` on a journal that cannot be read.

2. Rule format

Machine-readable definition: `docs/schemas/rule.schema.json`. A rule is JSON or YAML with these keys:

- `schema_version` (int, currently `1`), `rule_id`, `rule_version`, `name`.
- `scope.direction` — `"A_to_B"` or `"B_to_A"`; absent means either direction.
- `framing` — see below.
- `fields[]` — ordered list of fields.

Field keys: `name`, `offset`, `type`, `byte_order` (`"big"` default or `"little"`), `hypothesis: true` (the field's meaning is an assumption, not an observation), `expected: [v1, v2]` (allowed values; a value outside the list is `mismatched`), and `missing_ok: true` (the field may be absent — part of the stream may be a gap — without failing the message; it is then `uncovered`). The remaining keys depend on `type`.

Field types

Directly read from the message layout:

- Numeric `uint8, uint16, uint32, int8, int16, int32` — fixed width.
- `bytes` — requires `length`; value is the lowercase hex string.

- `enum` — requires an `enum` mapping of names to integer values.
- `bitmask` — a bit range inside one or more bytes: `bit_offset` (LSB-based) and `bit_length`. The storage width is `bit_offset + bit_length` rounded up to whole bytes, so a range may cross a byte boundary. `json { "name": "version", "offset": 0, "type": "bitmask", "bit_offset": 0, "bit_length": 2 } { "name": "mode", "offset": 0, "type": "bitmask", "bit_offset": 2, "bit_length": 2 }`
- `string` — `length` for a fixed-width string, or `terminated: true` for a NUL-terminated one (the terminator is included in the field span). `encoding` is `ascii` (default), `utf-8` or `latin-1`. Trailing NUL padding is stripped.
  `json { "name": "tag", "offset": 4, "type": "string", "length": 8, "encoding": "ascii" } { "name": "name", "offset": 12, "type": "string", "terminated": true }`
- `array` — repeated elements: `element_type` (a numeric type or `bytes`), `element_length` (for `bytes` elements) and a count given by `count` or by `count_field` (another decoded field). The value is a list. `json { "name": "values", "offset": 2, "type": "array", "element_type": "uint16", "count": 5 } { "name": "items", "offset": 4, "type": "array", "element_type": "bytes", "element_length": 2, "count_field": "item_count" }`
- `padding` — `length` bytes skipped when reading but kept in the message span (for alignment). Its value is `null` and it is `uncovered`; a message of only padding is `uncovered`.

Checked rather than extracted (the message span is still reserved):

- `checksum` — `algorithm` (`xor`, `sum`, `crc8`, `crc16`), `length` (defaults to the algorithm width: 1 byte, 2 for `crc16`), and the byte range the sum covers given by `start` and `end` (offsets relative to the message start; `end` defaults to the message length). The stored value is compared with the computed one; a difference is `mismatched`. `src.protocol.checksums.width_for(algorithm) -> int` returns that default width, and the helper itself is public so a caller can size a field without duplicating the table. `json { "name": "crc", "offset": 6, "type": "checksum", "algorithm": "crc16", "length": 2, "start": 0, "end": 6 }`
- `computed` — `expression` describes a value derived from other fields. The field's own bytes (`length`, default 1) are compared with the computed value. Expressions: `sum_of_values` (`fields`), `sum_of_lengths`, `xor` (`fields`, optional `value`), `const` (`value`), `add`/`sub`. Operands are referenced by field name; if any operand is missing the field is `incomplete`. `json { "name": "payload_length", "offset": 2, "type": "computed", "length": 2, "expression": { "op": "sum_of_lengths", "fields": ["body"] } } { "name": "total", "offset": 4, "type": "computed", "expression": { "op": "sum_of_values", "fields": ["a", "b"] } }`

Conditional (present only when a condition holds):

- `conditional` — `condition` is `{ "field": <name>, "op": ..., "value": ... }` with `op` in `eq`, `ne`, `bit_set`, `bit_clear`, `gt`, `lt`; the nested `field` object describes the value when the condition holds. When the condition is not met the value is `null` and the field is `uncovered` (it does not fail the message). `json { "name": "value_2", "type": "conditional", "condition": { "field": "flags", "op": "bit_set", "value": 1 }, "field": { "name": "value_2", "offset": 6, "type": "uint16" } }`

References between fields (`conditions`, `computed`, `count_field`) resolve by name against fields declared earlier in the list. A reference to an undeclared field is rejected when the rule is parsed.

Framing

`framing.type` is one of `length_prefixed`, `fixed_size`, `marker_based`, `manual`. Only `length_prefixed` has an ambiguity about what the length value counts; `length_covers` resolves it:

length_covers	What the length value counts	Message length
payload (default)	only the payload	length_offset + length_size + value
payload_and_length_field	payload plus the length field	length_offset + value
entire_message	the whole message	value

For length_prefixed, length_offset is the byte offset of the length value inside the message, length_size its width in bytes, byte_order its endianness.

Examples for the same declared value 6 with length_offset = 2, length_size = 2:

```
{ "type": "length_prefixed", "length_offset": 2, "length_size": 2, "length_covers": "payload" }
```

total length = 2 + 2 + 6 = 10 bytes.

```
{ "type": "length_prefixed", "length_offset": 2, "length_size": 2, "length_covers": "pay-
load_and_length_field" }
```

total length = 2 + 6 = 8 bytes.

```
{ "type": "length_prefixed", "length_offset": 2, "length_size": 2, "length_covers": "en-
tire_message" }
```

total length = 6 bytes.

The deprecated boolean length_includes_payload is still accepted for old rules: true maps to entire_message, false maps to payload. New rules should use length_covers.

fixed_size uses size. marker_based uses start_bytes and end_bytes (byte strings may be given as "hex:aa55", [0xaa, 0x55] or plain text) and include_markers. manual uses an explicit messages list of {offset, length}.

3. Result format

Machine-readable definition: docs/schemas/result.schema.json. The result is the apply output consumed by the GUI and by the report module:

```
{
  "contract_version": 1,
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "capture_id": "sha256:...",
  "messages": [
    { "offset": 0, "length": 8,
      "fields": { "command": 4, "parameter_value": 21 },
      "status": "matched",
      "provenance_range": { "offset": 0, "length": 8, "packets": [7] } }
  ],
  "summary": { "matched": 205, "mismatched": 12, "incomplete": 0,
               "ambiguous": 0, "uncovered": 0, "not_applicable": 0,
               "outdated": 0, "unknown": 0 }
}
```

Statuses

Message level:

- `matched` — framing and every declared field read consistently with the rule.
- `mismatched` — a field value contradicts the rule (for example an `expected` value was not seen).
- `incomplete` — the message or a field crosses a gap or the end of the stream.
- `ambiguous` — a field range overlaps bytes the reassembly reported as contradictory.
- `uncovered` — no declared field is present in this message.
- `not_applicable` — the rule scope excludes this direction.

Result level:

- `outdated` — the result was produced by an older rule version.
- `unknown` — reserved.

`matched` means "consistent with the rule over the messages that were tested". It is not proof; a wider corpus can contradict a rule, which is why counterexamples are collected.

4. Worked example

Rule (`rule.json`):

```
{
  "schema_version": 1,
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "name": "set_parameter_request",
  "scope": { "direction": "A_to_B" },
  "framing": { "type": "length_prefixed", "length_offset": 2, "length_size": 2,
               "byte_order": "big", "length_covers": "entire_message" },
  "fields": [
    { "name": "command", "offset": 0, "type": "uint8", "expected": [4] },
    { "name": "flags", "offset": 1, "type": "uint8" },
    { "name": "payload_length", "offset": 2, "type": "uint16", "byte_order": "big" },
    { "name": "parameter_value", "offset": 4, "type": "uint16", "byte_order": "big", "hypothes-
is": true }
  ]
}
```

Capture (`capture.json`, produced by the capture module) holds three 8-byte messages in session `s1`, direction `A_to_B`: two with `command = 4` and one with `command = 9`.

Programmatic use:

```
from src.protocol.stream import load_capture
from src.protocol.rule import load_rule
from src.protocol.engine import apply_rule
from src.protocol.result import build_result, write_result

capture = load_capture("capture.json")
rule = load_rule("rule.json")
stream = capture.stream("s1", "A_to_B")
messages = apply_rule(stream, rule, "s1", "A_to_B")
result = build_result(rule, capture.capture_hash, messages)
write_result(result, "result.json")
```

Command line:

```
python -m src.protocol.cli apply --rule rule.json --capture capture.json --out result.json
python -m src.protocol.cli verify --rule rule.json --capture capture.json --out report.json
```

`apply` output (abridged):

```
{
  "contract_version": 1,
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "capture_id": "sha256:...",
  "messages": [
    { "offset": 0, "length": 8, "fields": { "command": 4, "flags": 0,
      "payload_length": 8, "parameter_value": 21 }, "status": "matched" },
    { "offset": 8, "length": 8, "fields": { "command": 4, "flags": 0,
      "payload_length": 8, "parameter_value": 22 }, "status": "matched" },
    { "offset": 16, "length": 8, "fields": { "command": 9, "flags": 0,
      "payload_length": 8, "parameter_value": 23 }, "status": "mismatched" }
  ],
  "summary": { "matched": 2, "mismatched": 1, "incomplete": 0, "ambiguous": 0,
               "uncovered": 0, "not_applicable": 0, "outdated": 0, "unknown": 0 }
}
```

`verify` output lists the counterexamples alongside the counts:

```
{
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "total": 3,
  "counts": { "matched": 2, "mismatched": 1, "...": 0 },
  "contradictions": [
    { "status": "mismatched", "session_id": "s1", "direction": "A_to_B",
      "message_offset": 16, "message_length": 8,
      "reason": "value 9 not in expected [4]",
      "field_name": "command", "field_offset": 0, "field_length": 1,
      "bytes_hex": "0900000800170000",
      "provenance_range": { "offset": 16, "length": 1, "packets": [7] } }
  ],
  "summary": "2 messages matched, 1 counterexamples (1 mismatched)",
  "capture_id": "sha256:..."
}
```


5. Reading counterexamples

A `Counterexample` answers "which bytes disagree, and where do they come from".

- `message_offset` and `message_length` locate the message inside the directional stream (byte positions after reassembly).
- `field_name`, `field_offset`, `field_length` locate the offending field relative to the message start.
- `bytes_hex` is the raw message bytes. `bytes.fromhex(bytes_hex)` recovers them. In the example the message is `09 00 00 08 00 17 00 00` and the field is its first byte `09`.
- `provenance_range` ties the range back to the source: `{ "offset": start, "length": n, "packets": [packet_index, ...] }`. The `offset` and `length` are positions in the directional stream; `packets` are the capture packet indices (as reported by the capture module) that contributed those bytes. Use them to select the originating packets and to check the bytes against the source file.

A field-level counterexample carries the provenance of the field range, not of the whole message, so the GUI can highlight exactly the bytes that disagree.

6. Rule versions

- Editing a rule that changes framing or field reading creates a new version: `revised = rule.bump_version()` (optionally `rule.bump_version(fields=...)`).
- The engine never rewrites source bytes. Applying a revised rule to the same stream yields new results; the old ones are unchanged until marked.
- `ResultStore.add(rule, capture_id, payload)` stores a result tagged with the rule version that produced it.
- `ResultStore.mark_outdated(revised_rule, capture_id)` flips stored results whose version is older than the revised rule to `outdated`: each stored result gets `status = "outdated"`, every message inside its payload is retagged `outdated`, and its summary is zeroed with `outdated` set to the message count. This is how an old check is kept from being shown as current.
- `compare_reports(base_report, new_report)` returns a `VersionComparison` with `resolved` counterexamples (present before, gone now) and `introduced` counterexamples (new after the revision). The GUI uses it to show the effect of an edit.

7. Diagnostics

The capture contract reports two diagnostics that reach the engine through `Diagnostic.type` on a direction: `gap` and `ambiguity`. The engine turns them into statuses.

- `gap` (`{type: "gap", offset, length}`) — bytes were never observed. They are not zero-filled. A message or field whose range intersects a gap becomes `incomplete`.
- `ambiguity` — overlapping bytes disagrees between retransmissions. A device that is not ambiguous is irrelevant; when an ambiguous range intersects a message or field, that message becomes `ambiguous` rather than silently choosing one version.
- `incomplete` — also used when framing needs bytes past the end of the stream (a truncated tail). The message is reported with its observed length and `complete = false`.
- `uncovered` — a message where every declared field is absent by design (`missing_ok`). It carries no interpreted value, so it is neither matched nor mismatched.

The GUI should show `incomplete`, `ambiguous` and `uncovered` distinctly from `mismatched`: they describe a limitation of the data or the rule's reach, not a contradiction of a claim.

8. Metrics

`compute_metrics(report)` turns a verification report into numbers, and the CLI `metrics` command writes them as JSON. `RuleMetrics`:

- `total_messages`, `applicable_messages` (messages the rule scope covers), `sessions`, `matched`, `mismatched`, `counterexamples`;
- `coverage = (total - not_applicable) / total` — how much of the corpus the rule reaches;
- `precision = matched / (matched + mismatched)` — of the messages it interprets, how many agree with it;
- `fragmentation = total_messages / sessions` — average messages per session, a sign the framing actually split the stream;
- `unknown_ratio = unknown / total`;
- `uncovered_ratio` — bytes of the streams not consumed by any framed message, `(stream_bytes - covered_bytes) / stream_bytes`;
- `counterexample_density = counterexamples / applicable * 100`;
- `length_histogram` — message length to count;
- `type_histogram` — values of a `command` field to count, when such a field is extracted.

```
python -m src.protocol.cli metrics --report report.json --out metrics.json
```

9. Journal correlation

A journal is the researcher's own record of what happened during a capture, written as plain text separately from the bytes. Each non-comment line is either `<timestamp> | <action> | key=value | ...` or the positional `<timestamp> | <action> | <parameter> | <result>`; the two are told apart by whether the third field contains `=` (the positional form stores `parameter` and `result` as keys). The timestamp is epoch seconds or ISO-8601; `#` starts a comment, a line that does not start with a digit is prose, and blank lines are ignored. `parse_timestamp(text) -> float` is the parser behind that rule.

```
100.02 | set_temperature | value=21
100.31 | toggle_power
100.40 | read | temperature | 1595
```

`correlate(messages, entries, window_ms=500)` pairs each timestamped message with the nearest journal entry whose timestamp is within the window. A timestamp comes from the message's provenance (the earliest packet time), which is why the engine records `ts` on each `FieldResult.provenance_range` and on `MessageResult.timestamp`. The result is a `CorrelationReport`:

- `correlated` — one `Correlation` per paired message: the message offset and time, the entry, `delta_ms`, every decoded field value (`message_values`), the journal keys that match a field name, and `value_agreements`/`value_conflicts` (the decoded value against the journal parameter). A conflict is evidence the field does not mean what the journal says, and is kept, not dropped.
- `messages_without_timestamp`, `messages_without_entry` — messages that could not be paired, and why;
- `entries_without_message` — journal lines not followed by a message.

A correlation is a temporal suggestion, not proof: a nearby message with the intended value supports a field hypothesis; an empty window reports that no evidence was available. An entry is used at most once.

```
python -m src.protocol.cli correlate --rule rule.json --capture capture.json \
    --journal journal.md --window-ms 500 --out correlations.json
```

Request and response pairing

Journal correlation relates a message to the researcher's own notes. Pairing relates a message to another message: when a rule frames both directions, `build_turns(messages, session_id="", window_seconds=5.0)` orders the messages by timestamp and pairs each one with the next message in the opposite direction. It is in `src/hypothesis/pairing.py`.

Each `Turn` holds the request, the response (or `None`), the observed `gap_seconds`, a `confidence` in `[0, 1]`, and `basis`, the list of observations the confidence rests on. A message with no follower in the opposite direction is reported as unanswered with confidence `0`. Confidence is a fixed weighted sum — base `0.40`, opposite directions `0.15`, observed time order `0.15`, a reply within the window `0.10`, both messages framed cleanly `0.10`, both matched `0.05`, minus `0.20` when either message was framed weakly (`ambiguous`, `incomplete`, `unknown`) and `0.15` when a timestamp is missing. The weights are stated at the top of the module.

The number is a score over those observations, not a probability that the reply belongs to the request. Pairing never asserts ownership; it reports order, direction and timing, and leaves the meaning to the researcher.

```
python -m src.protocol.cli pair --rule rule.json --capture capture.json \
    --session s1 --window-ms 500 --out pairing.json
```

`build_turns_for_corpus(streams)` pairs within each session separately, so a reply from one connection is never matched to a request from another.

10. Alternatives and export

`analyze_field(rule, messages, field_name, correlations=None)` tests competing readings of one field and scores each by how many messages it agrees with. The candidates are `constant`, `counter`, `message_length`, `remaining_length`, a `checksum_<algo>_<range>` over the message body, `low_cardinality`, `alias_of_<other>` when the field mirrors another, `journal_<key>` when correlations are supplied, and four encoding readings for numeric fields: `endianness` (the counter reads little-endian while the rule says big-endian), `offset_shift_+1`/`offset_shift_-1` (a neighbouring byte is the real constant field), `xor_mask_<k>` (the values are masked with a constant), and `delta_encoding` (the field stores the

step since the previous message). The encoding readings are offered only when the declared big-endian values are not already the simplest sequence, so a plain counter is not reported as masked. Each `Alternative` carries `support`, `contradict`, `score = support / (support + contradict)` and up to a few `evidence` entries. `suggest_alternatives` runs it over every field marked `hypothesis: true`. The declared meaning is one candidate among them and may lose; the output is the evidence, not a verdict.

Four further readings summarize how a numeric field varies rather than what it means; they use only the standard library and take only the field's values. Each is also offered as a candidate when its condition holds:

- `entropy(field_values) -> float` — Shannon entropy in bits per value. More than 6 bits suggests a nonce, hash or random field; 3 to 6 bits a value that changes (counter or measurement); fewer than 2 bits an enum or flags. The candidate names are `entropy_random`, `entropy_parameter`, `entropy_enum`.
- `periodicity(field_values) -> int | None` — the smallest lag at which the autocorrelation is a clear local peak, or `None`. A monotone ramp or a plain counter returns `None`, because they correlate at every short lag. The candidate name is `periodicity`.
- `bit_pattern(field_values, bit_count=4) -> dict` — how stable the high-order bits are (`bit_count` in `(2, 4, 6)`): `verdict` is `flags_high` above 90 percent stable, `contradict` at or below 50 percent, else `inconclusive`. The candidate names are `bit_pattern_2`, `bit_pattern_4`, `bit_pattern_6`.
- `delta_correlation(field_values) -> dict` — the coefficient of variation of the first differences: `delta` below 0.1 (a fixed step or delta encoding), `contradict` above 0.5. The candidate name is `delta_correlation`.

Non-numeric values are out of scope for all four and are ignored.

```
python -m src.protocol.cli alternatives --rule rule.json --report report.json \
    --journal journal.md --field value --out alternatives.json
python -m src.protocol.cli alternatives --rule rule.json --report report.json \
    --all --out alternatives.json
```

`--field NAME` analyses one field; `--all` analyses every hypothesis field in the rule.

`export_kaitai(rule)` returns a Kaitai Struct `.ksy` document and `export_python(rule)` a standalone Python module with no dependency on this project. Both reproduce the framing and the field reads; the generated module's `parse_stream(data)` returns the same spans and values the engine does, and it checks checksums and computed fields itself. Verifying hypotheses, ambiguity and provenance stay in the engine.

```
python -m src.protocol.cli export --rule rule.json --format kaitai --out out/
python -m src.protocol.cli export --rule rule.json --format python --out parser.py
```

11. GUI integration

The window in `src.ui` is a thin consumer of this module. It never parses a capture and never re-implements framing: it loads a normalized capture, hands a direction and a rule to `apply_rule`, and renders the result.

What the GUI expects

- A normalized capture, either a file (`load_capture`) or parsed JSON (`capture_from_dict`). The reference exports under `tests/corpus/reference_export/` are exactly this shape.
- A rule, either a file (`load_rule`) or edited text (`load_rule_text` + `parse_rule`).

What the GUI calls

- `CaptureModel.from_file(path)` wraps `capture_from_dict` and satisfies the session list and diagnostics from `sessions` and `directions[*].diagnostics`.
- `RuleModel.apply` calls `apply_rule(stream, rule, session_id, direction)` and keeps the `MessageResult` list. `counts` and `counterexamples` are read off it.
- `verify_on_corpus(rule, streams)` backs the "apply to whole capture" command; `CorpusStream.from_bytes(data, session_id, direction)` builds its input.
- Byte provenance for the status bar comes from `DirectionalStream.provenance` (each `Hole` carries `packet_index`, `seq`, `ts`), or from `FieldResult.provenance_range` for the field under the pointer.

Field-level details the GUI relies on

- `FieldResult.field_offset` and `field_length` locate a byte range inside its message; the hex view highlights `message.offset + field_offset` onward.
- `FieldResult.hypothesis` marks an assumption; the messages table labels it.
- `MessageResult.status` and `bytes_hex` drive the per-message row and the counterexample list.
- `MessageResult.provenance_range` is not a field of the message; provenance is read from the message's fields (`fields[0].provenance_range` when the message has fields). This is what `src.ui.validation_view` and the integration tests use.

Example

Open the reference capture, apply rule v1, and read the result:

```
from src.protocol.engine import apply_rule
from src.protocol.rule import load_rule
from src.protocol.stream import load_capture

capture = load_capture("tests/corpus/reference_export/corpus_capture_01.normalized.json")
rule = load_rule("examples/corpus_rule_v1.json")
stream = capture.stream("s1", "A_to_B")
messages = apply_rule(stream, rule, "s1", "A_to_B")
matched = sum(m.status.value == "matched" for m in messages)
```

To produce the contract result the report and the export use:

```
from src.protocol.result import build_result

payload = build_result(rule, capture.capture_hash, messages).to_dict()
```

The result validates against `docs/schemas/result.schema.json`. A byte the user clicks is reported as `session + direction + offset + packet_index + seq + ts`, which is enough to find it in the original capture.

12. Limitations

The rule model is flat by design: arrays repeat a single fixed-width element and `computed`/`conditional` reference other declared fields, but there are no nested sub-records, no variable-length elements, and no layout that changes wholesale per variant. A protocol variant is described by a separate rule scoped to it. Anything requiring per-message scripting belongs in a caller that reads `apply_rule` output. Fields are byte-aligned except `bitmask`. Encrypted or compressed payloads are not decoded.

These fit the case, where the goal is a checkable interpretation of a stream rather than a complete protocol description; they can be extended without changing the result contract.